Major Assignment – CS771

Group 24 PJADS

Jayant Mitawa
230504
jayantm23@iitk.ac.in

Sujal Aggarwal
231047
asujal23@iitk.ac.in

Divyansh Bansal
230381
divyanshb23@iitk.ac.in

Priyen Parmar
230735
parmari23@iitk.ac.in

Aryan Deo
230213
aryandeo23@iitk.ac.in

Abstract

This assignment addresses two independent challenges involving kernel methods and hardware security. In the first part, we study a semi-parametric regression model for predicting the number of staff required to review online video submissions. The given model includes both a parametric dependence on video length and a non-parametric dependence on popularity and difficulty. To enable training using kernel ridge regression, we derive an equivalent pure kernel representation by augmenting the feature space, yielding a valid Mercer kernel of the form $\tilde{K}((x_1, z_1), (x_2, z_2)) = x_1 x_2 (z_1^\top z_2 + c)^d + 1$. We evaluate multiple polynomial degrees and offset hyperparameters on the provided dataset, achieving a best public-test performance of $R^2 = 0.97004$ using $(d = 3, c = 1.6)$. This kernel is then implemented efficiently in a vectorized form to support large-scale Gram matrix computation.

In the second part, we investigate the inversion of XOR Arbiter Physical Unclonable Functions (PUFs). Each PUF evaluation corresponds to a linear response model in a 1089-dimensional feature space, and the objective is to recover the underlying delay parameters from a single learned model vector. Using the structure of the feature expansion, we derive a principled decoding strategy that decomposes the model into eight 32-dimensional delay vectors, which are returned by the `my_decode` implementation. Together, these two components demonstrate the use of kernel methods for semi-parametric learning and the exploitation of algebraic structure for PUF inversion.

Problem 1.1: Hiring Problem

Problem Formulation

We aim to model the number of staff members y as a function of video length $x \in \mathbb{R}$ and content features $\mathbf{z} \in \mathbb{R}^2$. The relationship is given by the semi-parametric equation:

$$y = (\mathbf{p}^\top \phi(\mathbf{z})) \cdot x + b \tag{1}$$

where:

- x is the scalar video length (linear component).
- $\mathbf{z}$ represents popularity and difficulty (non-linear component).

- $\phi : \mathbb{R}^2 \to \mathcal{H}$ is a feature map corresponding to a polynomial kernel $K_z(\mathbf{z}_1, \mathbf{z}_2) = (\mathbf{z}_1^\top \mathbf{z}_2 + c)^d$.
- $b \in \mathbb{R}$ is a scalar bias term.

Standard Kernel Ridge Regression (KRR) in `sklearn` assumes a model of the form $f(\cdot) = \mathbf{w}^\top \Psi(\cdot)$ without an explicit bias term for the kernel space. Therefore, we must engineer a composite kernel $\tilde{K}$ and a feature map ψ such that the standard KRR formulation captures the structure of Equation (1).

Part 1: Deriving the Pure Kernel

To utilize `sklearn.kernel_ridge`, we define a new feature map $\psi(x, \mathbf{z})$ that encapsulates the interaction between x and $\phi(\mathbf{z})$, as well as the bias term.

Feature Map Construction

We propose the augmented feature map $\psi : \mathbb{R} \times \mathbb{R}^2 \to \tilde{\mathcal{H}}$:

$$\psi(x, \mathbf{z}) = \begin{bmatrix} x \cdot \phi(\mathbf{z}) \\ 1 \end{bmatrix} \tag{2}$$

Here, the first component $x \cdot \phi(\mathbf{z})$ captures the multiplicative relationship between the video length and the non-linear features of $\mathbf{z}$. The second component, the constant 1, accounts for the bias term b.

Let the learnable parameter vector in this augmented space be $\tilde{\mathbf{p}} = [\mathbf{p}^\top, b]^\top$. The dot product in this space yields:

$$\begin{aligned} \tilde{\mathbf{p}}^\top \psi(x, \mathbf{z}) &= \begin{bmatrix} \mathbf{p}^\top & b \end{bmatrix} \begin{bmatrix} x \cdot \phi(\mathbf{z}) \\ 1 \end{bmatrix} \\ &= x \cdot \mathbf{p}^\top \phi(\mathbf{z}) + b \cdot 1 \\ &= (\mathbf{p}^\top \phi(\mathbf{z}))x + b \end{aligned}$$

This exactly recovers the desired semi-parametric model structure.

Kernel Derivation

The kernel function $\tilde{K}$ corresponding to ψ is defined as the inner product of the feature maps for two data points $(x_i, \mathbf{z}_i)$ and $(x_j, \mathbf{z}_j)$:

$$\begin{aligned} \tilde{K}((x_i, \mathbf{z}_i), (x_j, \mathbf{z}_j)) &= \langle \psi(x_i, \mathbf{z}_i), \psi(x_j, \mathbf{z}_j) \rangle \\ &= (x_i \phi(\mathbf{z}_i))^\top (x_j \phi(\mathbf{z}_j)) + (1)(1) \\ &= x_i x_j \left(\phi(\mathbf{z}_i)^\top \phi(\mathbf{z}_j) \right) + 1 \end{aligned} \tag{3}$$

We are given that the kernel on $\mathbf{z}$ is a polynomial kernel $K_z(\mathbf{z}_i, \mathbf{z}_j) = \phi(\mathbf{z}_i)^\top \phi(\mathbf{z}_j) = (\mathbf{z}_i^\top \mathbf{z}_j + c)^d$. Substituting this back, we obtain the final kernel form:

$$\boxed{\tilde{K}((x_i, \mathbf{z}_i), (x_j, \mathbf{z}_j)) = x_i x_j (\mathbf{z}_i^\top \mathbf{z}_j + c)^d + 1} \tag{4}$$

Validity: This kernel is a valid Mercer kernel because it is constructed from the product of two valid kernels (a linear kernel on x and a polynomial kernel on $\mathbf{z}$) plus a valid constant kernel.

Part 2: Hyperparameter Tuning

To identify the optimal hyperparameters for the polynomial kernel $K(m, n) = (m^\top n + c)^d$, we performed a four-stage exhaustive grid search. We tuned the degree d and coefficient c to maximize the R^2 score. Below are the complete tables containing every data point tested during this process.

Step 1: Initial Coarse Grid Search

We first swept degrees $d \in [2, 10]$ with logarithmic coefficients. The full results are listed in Table 1.

Table 1: **Complete Data: Coarse Grid Search**

Degree (d)	**Coefficient** (c)	R^2 **Score**	**Degree** (d)	**Coefficient** (c)	R^2 **Score**
2	0.01	0.94842	7	0.01	0.32490
2	0.1	0.96995	7	0.1	0.56832
2	1	0.96991	7	1	0.96966
2	10	0.96991	7	10	0.96953
2	100	0.96991	7	100	0.46438
3	0.01	0.81095	8	0.01	0.27504
3	0.1	0.96955	8	0.1	0.46421
3	1	0.96993	8	1	0.96959
3	10	0.96993	8	10	0.96952
3	100	0.96993	8	100	0.46421
4	0.01	0.62899	9	0.01	0.23594
4	0.1	0.95400	9	0.1	0.39012
4	1	0.96992	9	1	0.96956
4	10	0.96991	9	10	0.96950
4	100	0.96991	9	100	0.46408
5	0.01	0.48892	10	0.01	0.20365
5	0.1	0.86655	10	0.1	0.33613
5	1	0.96986	10	1	0.96954
5	10	0.96977	10	10	0.96912
5	100	0.96965	10	100	0.46696

Step 2: Linear Range Search ($c \in [0.1, 3.0]$)

We then narrowed the degree to $d \in \{2, 3, 4\}$ and tested c in increments of 0.5. Every value tested is shown below.

Table 2: **Complete Data: Linear Range Search**

Degree (d)	**Coefficient** (c)	R^2 **Score**
2	0.1	0.96995
2	0.5	0.96992
2	1	0.96991
2	1.5	0.96991
2	2.0	0.96991
2	2.5	0.96991
2	3	0.96991
3	0.1	0.96955
3	0.5	0.96993
3	1	0.96993
3	1.5	0.96993
3	2.0	0.96993
3	2.5	0.96993
3	3	0.96993
4	0.1	0.95400
4	0.5	0.96993
4	1	0.96992
4	1.5	0.96991
4	2.0	0.96991
4	2.5	0.96991
4	3	0.96991

Step 3: Dense Refinement ($c \in [0.1, 0.5]$)

We refined the search for $d \in \{2, 3, 4\}$ with c in smaller steps of 0.05.

Table 3: **Complete Data: Dense Refinement**

Degree (d)	**Coefficient** (c)	R^2 **Score**
2	0.1	0.96995
2	0.15	0.96994
2	0.2	0.96993
2	0.25	0.96992
2	0.3	0.96992
2	0.35	0.96992
2	0.4	0.96992
2	0.45	0.96992
2	0.5	0.96992
3	0.1	0.96955
3	0.15	0.97004
3	0.2	0.97002
3	0.25	0.96998
3	0.3	0.96996
3	0.35	0.96995
3	0.4	0.96994
3	0.45	0.96994
3	0.5	0.96993
4	0.1	0.95400
4	0.15	0.96809
4	0.2	0.96984
4	0.25	0.97001
4	0.3	0.97000
4	0.35	0.96997
4	0.4	0.96995
4	0.45	0.96994
4	0.5	0.96993

Step 4: Final Fine-Tuning ($d = 3$)

Finally, we performed a high-resolution sweep for $d = 3$ around the peak performance region.

Table 4: **Complete Data: Final Fine-Tuning for $d = 3$**

Degree (d)	**Coefficient (c)**	**R^2 Score**
3	0.1	0.96955
3	0.11	0.96979
3	0.12	0.96992
3	0.13	0.96999
3	0.14	0.97002
3	0.15	**0.97004**
3	0.16	**0.97004**
3	0.17	**0.97004**
3	0.18	0.97003
3	0.19	0.97002
3	0.2	0.97002

Final Selection: Based on the trend analysis and the peak values observed in Table 4, we selected the following hyperparameters:

$$\text{Optimal Degree } d = 3, \quad \text{Optimal Coefficient } c = 0.16.$$

Analysis and Selection

Based on the results above, we observe the following trends:

1. **Effect of c:** When $c = 0$, the kernel becomes homogeneous. The performance is significantly lower for $c = 0$ across all degrees, suggesting that the inhomogeneous component (the bias within the polynomial) is crucial for modeling the interaction between popularity and difficulty.
2. **Effect of d:** Higher degrees generally increase model expressivity. However, beyond a certain degree, the gain in R^2 saturates.

Final Choice: We select the configuration that maximizes performance while maintaining the lowest model complexity (lowest degree) to prevent potential overfitting on unseen data.

$$d^* = \mathbf{3} \quad \text{and} \quad c^* = \mathbf{0.16}$$

The selected configuration achieves an R^2 score of approximately $\mathbf{0.97004}$ on the dataset.

Problem 1.2 : Delay Recovery Problem

Part 4 : Methodology

Setup : Each single arbiter PUF (with $k = 32$ challenge bits) has k multiplexers indexed $i = 0, \ldots, 31$. Each multiplexer has four delays, which we denote for the first arbiter as

$$\mathbf{a} = (a_0, \ldots, a_{31}), \quad \mathbf{b} = (b_0, \ldots, b_{31}), \quad \mathbf{c} = (c_0, \ldots, c_{31}), \quad \mathbf{d} = (d_0, \ldots, d_{31}),$$

and for the second arbiter as

$$\mathbf{p} = (p_0, \ldots, p_{31}), \quad \mathbf{q} = (q_0, \ldots, q_{31}), \quad \mathbf{r} = (r_0, \ldots, r_{31}), \quad \mathbf{s} = (s_0, \ldots, s_{31}).$$

For one arbiter, define two length-k sequences α_i and β_i by a linear mapping of the four per-multiplexer delays:

$$\alpha_i = \tfrac{1}{2}\big(x_i^{(1)} - x_i^{(2)} + x_i^{(3)} - x_i^{(4)}\big), \tag{5}$$

$$\beta_i = \tfrac{1}{2}\big(x_i^{(1)} - x_i^{(2)} - x_i^{(3)} + x_i^{(4)}\big), \tag{6}$$

where $(x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)})$ stands for (a, b, c, d) (or (p, q, r, s)) for that arbiter.

The arbiter linear model for a single arbiter is the $(k+1)$-vector $u \in \mathbb{R}^{k+1}$ given by

$$u_0 = \alpha_0, \tag{7}$$

$$u_i = \alpha_i + \beta_{i-1} \quad (1 \le i \le k-1), \tag{8}$$

$$u_k = \beta_{k-1}. \tag{9}$$

XOR arbiter linear model. If $u \in \mathbb{R}^{k+1}$ and $v \in \mathbb{R}^{k+1}$ are the linear models of the two component arbiters, the XOR-arbiter linear model is the Kronecker (outer) product

$$W = u\, v^\top \in \mathbb{R}^{(k+1)\times(k+1)}.$$

Given a single flattened $w \in \mathbb{R}^{(k+1)^2}$ (here $(k+1)^2 = 33^2 = 1089$), reshape to $W \in \mathbb{R}^{33\times 33}$.

Method Outline

1. Reshape $w \mapsto W$ of size 33×33.
2. Compute rank-1 approximation of W via SVD:
   $$W \approx \sigma_1\, \mathbf{u}_1 \mathbf{v}_1^\top,$$
   where σ_1 is the largest singular value and $\mathbf{u}_1, \mathbf{v}_1$ are the corresponding singular vectors. Set
   $$\hat{u} = \sqrt{\sigma_1}\, \mathbf{u}_1, \qquad \hat{v} = \sqrt{\sigma_1}\, \mathbf{v}_1.$$
   (Optionally flip signs $(\hat{u}, \hat{v}) \leftarrow (-\hat{u}, -\hat{v})$ so that $\hat{u}$ has mostly nonnegative entries.)
3. Build the linear mapping matrix $A \in \mathbb{R}^{33\times 128}$ that maps the 128 per-arbiter delays vector
   $$x = [x_0^{(1)}, \ldots, x_{31}^{(1)}, x_0^{(2)}, \ldots, x_{31}^{(2)}, x_0^{(3)}, \ldots, x_{31}^{(3)}, x_0^{(4)}, \ldots, x_{31}^{(4)}]^\top$$
   to the 33-element model $u = Ax$, using the definitions of α_i, β_i and the relations for u_j above. (Each row of A contains $\pm\frac{1}{2}$ in the columns corresponding to the involved delays.)
4. Recover nonnegative delays for the first arbiter by solving the constrained least-squares
   $$x^{(1)} = \arg\min_{x \ge 0} \|Ax - \hat{u}\|_2^2,$$
   and for the second arbiter
   $$x^{(2)} = \arg\min_{x \ge 0} \|Ax - \hat{v}\|_2^2.$$
   These are nonnegative least-squares problems and can be solved with standard solvers (bounded least squares, NNLS).
5. Construct $u_{\text{rec}} = Ax^{(1)}$, $v_{\text{rec}} = Ax^{(2)}$ and reconstruct $W_{\text{rec}} = u_{\text{rec}} v_{\text{rec}}^\top$ then flatten to $w_{\text{rec}} \in \mathbb{R}^{1089}$.
6. Compute evaluation metrics: runtime and Euclidean reconstruction error $\|w_{\text{rec}} - w\|_2$.

Remarks

- The SVD step finds approximate factor vectors $\hat{u}, \hat{v}$ even when W is not exactly rank-1 (noisy/leaked model). The NNLS step finds nonnegative per-multiplexer delays consistent (in least-squares sense) with the $\hat{u}, \hat{v}$ vectors.
- The mapping A is fixed and sparse; building it carefully ensures the optimization is small (33×128).

References

- CS771 Lecture Notes
- Lim, D., Lee, J. W., Gassend, B., Suh, G. E., van Dijk, M., & Devadas, S. (2005). Extracting secret keys from integrated circuits. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 13(10), 1200–1205.
- Rührmair, U., Sehnke, F., Sölter, J., Dror, G., Devadas, S., & Schmidhuber, J. (2010). Modeling attacks on physical unclonable functions. Proceedings of the 17th ACM Conference on Computer and Communications Security, 237–249.